Software Development Life Cycle (SDLC) Models - Stages Overview

1. Waterfall Model

Stage	Description
Requirements Analysis	Gather and document all system requirements from stakeholders
System Design	Create architecture and detailed design specifications
Implementation	Write code and develop system components
Testing	Verify system functionality and fix defects
Deployment	Release system to production environment
Maintenance	Provide ongoing support, bug fixes, and updates

Note: Best for stable requirements and well-defined projects. Suitable for medium to large projects. Use when requirements are clear, fixed timeline/budget, and documentation is mandatory.

---

2. Spiral Model

Phase	Activities
Planning	Identify objectives, alternatives, and constraints for iteration
Risk Analysis	Analyze risks, evaluate alternatives, develop mitigation strategies
Engineering	Develop and test product increment
Evaluation	Review results with stakeholders, plan next iteration

Note: Best for large, high-risk, complex projects. Use when requirements are uncertain and risk management is critical. Requires experienced team and larger budget.

---

3. Iterative Model

Stage	Description
Initial Planning	Define overall scope and high-level requirements
Requirements	Specify requirements for current iteration
Analysis and Design	Create design for current iteration features
Implementation	Code planned features for this iteration
Testing	Test implemented features and fix defects
Deployment	Release iteration output to users
Evaluation	Gather feedback and assess results

Stage	Description
Refinement	Incorporate feedback and prepare next iteration

Note: Best for projects with evolving requirements. Suitable for small to large projects. Use when working system needed early and feedback drives development.

---

4. Incremental Model

Stage	Description
Requirements Analysis	Gather complete system requirements upfront
System Design	Create overall architecture and identify modules
Increment Planning	Divide system into prioritized builds
Increment Design	Design specific increment in detail
Increment Implementation	Develop current increment
Increment Testing	Test current increment thoroughly
Integration	Integrate new increment with previous ones
Deployment	Deploy completed increment
Validation	Verify increment meets requirements

Note: Best for large projects with clear modules. Suitable for medium to large projects. Use when core features needed first and staged rollout preferred.

---

5. Build and Fix Model

Stage	Description
Initial Concept	Basic understanding with minimal requirements
Build	Start coding immediately without planning
Test and Fix	Test code and fix discovered bugs
Modify	Make changes based on feedback or issues
Repeat	Continue building, testing, fixing without formal process

Note: Only for very small throwaway projects or quick prototypes. High technical debt and poor quality. Avoid for production systems.

---

6. DevOps Model

Stage	Description
Plan	Define requirements, features, and goals
Code	Write source code with version control
Build	Compile code automatically into builds
Test	Execute automated tests continuously
Release	Prepare and package software for deployment
Deploy	Automate deployment to environments
Operate	Manage application in production
Monitor	Track performance, logs, and feedback

Note: Best for continuous delivery environments and cloud applications. Suitable for small to large projects. Use when frequent releases needed and automation culture exists.

---

7. V-Model (Verification and Validation Model)

Development Phase	Corresponding Test Phase
Requirements Analysis	Acceptance Testing
System Design	System Testing
High-Level Design	Integration Testing
Low-Level Design	Unit Testing
Coding (at bottom vertex)	-

Process: Left side descends through development, coding at bottom, right side ascends through corresponding tests.

Note: Best for projects requiring strict validation and safety-critical systems. Suitable for medium to large projects. Use when quality is critical and traceability required.

---

8. Agile Model

Stage	Description
Requirements Gathering	Create product backlog with prioritized user stories
Sprint Planning	Select backlog items and define sprint goals
Design	Create just-enough design for sprint features
Development	Implement features with continuous integration
Testing	Perform continuous testing throughout development

Stage	Description
Review	Demonstrate working software to stakeholders
Retrospective	Reflect on process and identify improvements
Deployment	Release potentially shippable product increment

Process: Repeat cycles every 1-4 weeks with continuous stakeholder collaboration.

Note: Best for evolving requirements and dynamic environments. Suitable for small to large projects. Use when requirements change frequently and customer collaboration is available.

Comparison Summary

Model	Key Advantages	Main Limitations
Waterfall	Clear structure, thorough documentation, defined milestones	Inflexible to changes, late testing, customer sees product late
Spiral	Strong risk management, flexible, early problem identification	Complex to manage, expensive, requires expertise
Iterative	Progressive refinement, working version early, reduced risk	Can be time-consuming, scope creep risk
Incremental	Early partial delivery, easier testing, parallel development	Requires good architecture, integration challenges
Build and Fix	Minimal overhead, quick start	Poor quality, unmaintainable, high technical debt
DevOps	Fast deployment, automation, quick feedback, reliability	Requires cultural change, tooling investment
V-Model	Early test planning, high quality, clear traceability	Rigid, expensive to change, sequential
Agile	Highly adaptable, quick delivery, continuous feedback	Requires skilled team, less predictability, minimal documentation

---

Agile Framework Details

Popular Agile Frameworks

Scrum:

- Sprint duration: 1-4 weeks (typically 2 weeks)
- Roles: Product Owner, Scrum Master, Development Team
- Ceremonies: Sprint Planning, Daily Standup, Sprint Review, Sprint Retrospective

- Artifacts: Product Backlog, Sprint Backlog, Increment

Kanban:

- Continuous flow without fixed iterations
- Visualize workflow on Kanban board
- Limit work in progress (WIP limits)
- Focus on cycle time and throughput

Extreme Programming (XP):

- Practices: Pair programming, Test-Driven Development (TDD), Continuous Integration
- Short development cycles
- Frequent releases
- Customer involvement in development

Lean Software Development:

- Eliminate waste
- Amplify learning
- Decide as late as possible
- Deliver as fast as possible
- Build quality in

Agile Principles

1. Customer satisfaction through early and continuous delivery
2. Welcome changing requirements, even late in development
3. Deliver working software frequently (weeks rather than months)
4. Business people and developers work together daily
5. Build projects around motivated individuals
6. Face-to-face conversation most efficient communication method
7. Working software is primary measure of progress
8. Sustainable development pace
9. Continuous attention to technical excellence
10. Simplicity - maximizing work not done
11. Self-organizing teams produce best results
12. Regular team reflection and adjustment

Agile Artifacts

- **Product Backlog:** Prioritized list of features, enhancements, and fixes
- **Sprint Backlog:** Items selected for current sprint with task breakdown
- **User Stories:** Feature descriptions from user perspective (As a [role], I want [feature], so that [benefit])
- **Burndown Chart:** Visual representation of work remaining vs time
- **Definition of Done:** Shared understanding of completion criteria
- **Velocity:** Measure of work completed per sprint

Agile Ceremonies

- **Sprint Planning:** Team selects work for sprint, creates sprint goal (2-4 hours for 2-week sprint)
- **Daily Standup:** 15-minute daily sync (What I did, what I'll do, blockers)
- **Sprint Review:** Demonstrate completed work to stakeholders (1-2 hours)
- **Sprint Retrospective:** Team reflects on process improvements (1-1.5 hours)
- **Backlog Refinement:** Ongoing activity to clarify and estimate backlog items

When to Use Agile

- Requirements are expected to evolve
- Need to deliver value quickly and continuously
- Customer/stakeholder available for regular collaboration
- Team is cross-functional and experienced
- Project benefits from iterative feedback
- Innovation and experimentation valued
- Market conditions change rapidly

When NOT to Use Agile

- Requirements are completely fixed and stable
- Regulatory environment requires extensive upfront documentation
- Team lacks experience or training in Agile
- Customer unavailable for collaboration
- Project has hard deadline with no flexibility
- Organization culture resistant to change